

Security Audit

Report for synclub - contracts

Date: September 11, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About Target Contracts	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Security Issues	2
1.3.2 Additional Recommendation	2
1.4 Security Model	3
Chapter 2 Findings	4
2.1 Security Issue	5
2.1.1 Potential DoS due to malicious vote delegations	5
2.1.2 Potential DoS due to lack of zero address check	6
2.1.3 Lack of whitelist checks for the input <code>dstValidator</code> in the function <code>redelegate()</code>	7
2.1.4 Lack of using the modifier <code>whenNotPaused</code>	7
2.1.5 Improper return value in the function <code>binarySearchCoveredMaxIndex()</code>	8
2.1.6 Potential lock of funds	8
2.2 Recommendation	9
2.2.1 Remove redundant code	9
2.2.2 Replace improper errors	10
2.2.3 Rename the variable <code>maxBufferSizePct</code>	11
2.2.4 Add zero value checks	11
2.2.5 Unify checks in the functions <code>instantWithdraw()</code> and <code>requestWithdraw()</code>	12
2.3 Note	13
2.3.1 The project should set the variable <code>instantWithdrawFeeRate</code> properly	13
2.3.2 Ensure the rationality of the variable <code>instantWithdrawFeeRate</code>	14
2.3.3 Contract deployment chains	15
2.3.4 Potential centralization risks	15
2.3.5 OpenZeppelin initializable upgrade migration risks	15
2.3.6 Proxy deployment and implementation binding should be atomic	15
2.3.7 Ensure that all <code>BOTs</code> are working properly	16

Report Manifest

Item	Description
Client	Lista - DAO
Target	synclub - contracts

Version History

Version	Date	Description
1.0	September 11, 2025	First release

Signature

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository¹ of synclub-contracts of Lista-DAO.

The synclub-contracts of Lista-DAO are designed to provide liquid staking functionality for BNBs on BSC by interacting with BSC's native StakeHub contract. Specifically, users deposit BNBs and receive corresponding shares in the form of slisBNB tokens. The contract ListaStakeManager then delegates the users' BNBs to whitelisted validators in the StakeHub contract to earn staking rewards. In addition, the contract ListaStakeManager introduces a buffer pool, which is designed to support instant withdrawals.

Note this audit only focuses on the smart contracts in the following directories/files:

- contracts/ListaStakeManager.sol

Other files are not within the scope of the audit. Additionally, all dependencies of the smart contracts within the audit scope are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version ([Version 1](#)), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version ([Version 0](#)), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
synclub-contracts	Version 1	a3524827a96be797151d154669c3cb023b6615d3
	Version 2	c1b83a6f5bd411222ad058d76872b34eeb37b45f

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any war-

¹<https://github.com/lista-dao/synclub-contracts>

ranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

- * Access control
- * Permission management
- * Whitelist and blacklist mechanisms
- * Initialization consistency
- * Improper use of the proxy system
- * Reentrancy
- * Denial of Service (DoS)
- * Untrusted external call and control flow
- * Exception handling
- * Data handling and flow
- * Events operation
- * Error-prone randomness
- * Oracle security
- * Business logic correctness
- * Semantic and functional consistency
- * Emergency mechanism
- * Economic and incentive impact

1.3.2 Additional Recommendation

- * Gas optimization

* Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology² and Common Weakness Enumeration³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we found **six** potential security issues. Besides, we have **five** recommendations and **seven** notes.

- Medium Risk: 1
- Low Risk: 5
- Recommendation: 5
- Note: 7

ID	Severity	Description	Category	Status
1	Medium	Potential DoS due to malicious vote delegations	Security Issue	Fixed
2	Low	Potential DoS due to lack of zero address check	Security Issue	Fixed
3	Low	Lack of whitelist checks for the input <code>dstValidator</code> in the function <code>redelegate()</code>	Security Issue	Fixed
4	Low	Lack of using the modifier <code>whenNotPaused</code>	Security Issue	Fixed
5	Low	Improper return value in the function <code>binarySearchCoveredMaxIndex()</code>	Security Issue	Confirmed
6	Low	Potential lock of funds	Security Issue	Confirmed
7	-	Remove redundant code	Recommendation	Partially Fixed
8	-	Replace improper errors	Recommendation	Fixed
9	-	Rename the variable <code>maxBufferSizePct</code>	Recommendation	Fixed
10	-	Add zero value checks	Recommendation	Partially Fixed
11	-	Unify checks in the functions <code>instantWithdraw()</code> and <code>requestWithdraw()</code>	Recommendation	Fixed
12	-	The project should set the variable <code>instantWithdrawFeeRate</code> properly	Note	-
13	-	Ensure the rationality of the variable <code>instantWithdrawFeeRate</code>	Note	-
14	-	Contract deployment chains	Note	-
15	-	Potential centralization risks	Note	-
16	-	OpenZeppelin initializable upgrade migration risks	Note	-
17	-	Proxy deployment and implementation binding should be atomic	Note	-
18	-	Ensure that all <code>BOTs</code> are working properly	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Potential DoS due to malicious vote delegations

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `ListaStakeManager`, the function `delegateVoteTo()` allows the admin to delegate the voting power of the contract `ListaStakeManager` to other operators in the contract `STAKE_HUB`. After the invocation of `govToken.delegates(address(this))`, the function `delegateVoteTo()` checks that the contract `ListaStakeManager` has no remaining voting power (Line 520). However, this design is improper as malicious users can delegate voting power to the contract `ListaStakeManager`, which can not be removed in the invocation of `govToken.delegates(address(this))`. As a result, the function `delegateVoteTo()` may suffer a DoS issue due to malicious vote delegations.

```
503 function delegateVoteTo(address _delegateTo) external override onlyRole(DEFAULT_ADMIN_ROLE) {
504     if (_delegateTo == address(0)) revert ErrorsLib.ZeroAddress();
505
506     IVotesUpgradeable govToken = IVotesUpgradeable(GOV_BNB);
507
508     address currentDelegatee = govToken.delegates(address(this));
509     require(currentDelegatee != _delegateTo, "Already Delegated");
510
511     uint256 balance = IERC20Upgradeable(GOV_BNB).balanceOf(address(this));
512
513     uint256 newVotePower = govToken.getVotes(_delegateTo);
514     uint256 currentVotePower = govToken.getVotes(currentDelegatee);
515     govToken.delegate(_delegateTo);
516     require(govToken.delegates(address(this)) == _delegateTo, "Delegation Failed");
517
518     // Check voting power moved correctly
519     if (_delegateTo != address(this)) {
520         require(govToken.getVotes(address(this)) == 0, "Invalid Delegation");
521         uint256 currDelegateeChange = currentVotePower - govToken.getVotes(currentDelegatee);
522         uint256 newDelegateeChange = govToken.getVotes(_delegateTo) - newVotePower;
523
524         require(currDelegateeChange == newDelegateeChange && balance == currDelegateeChange, "
                    Invalid Change");
525     } else {
526         require(govToken.getVotes(address(this)) == balance, "Self-delegation Failed");
527     }
528
529     emit DelegateVoteTo(_delegateTo, balance);
530 }
```

Listing 2.1: contracts/ListaStakeManager.sol

Impact The function `delegateVoteTo()` may suffer a DoS issue due to malicious vote delegations.

Suggestion Revise the logic accordingly.

2.1.2 Potential DoS due to lack of zero address check

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract [ListaStakeManager](#), the function [syncCredits\(\)](#) is responsible for synchronizing and storing the validator's credit contract (i.e., the variable [credit](#)). However, the function [syncCredits\(\)](#) lacks conducting the zero address check for the fetched credit contract, adding a zero address to the array [creditContracts](#). The lack of zero address checks leads the function [getTotalBnbInValidators\(\)](#) to revert (Line 969). As a result, the function [compoundRewards\(\)](#), which invokes the function [getTotalBnbInValidators\(\)](#), may suffer a temporary DoS issue due to adding the zero address to the array [creditContracts](#).

```
584 function syncCredits(address _validator, bool toRemove) internal {
585     address credit = IStakeHub(STAKE_HUB).getValidatorCreditContract(_validator);
586     if (toRemove) {
587         delete creditStates[credit];
588
589         for (uint256 i = 0; i < creditContracts.length; i++) {
590             if (creditContracts[i] == credit) {
591                 creditContracts[i] = creditContracts[creditContracts.length - 1];
592                 creditContracts.pop();
593                 break;
594             }
595         }
596         emit SyncCreditContract(_validator, credit, toRemove);
597         return;
598     } else if (creditStates[credit]) {
599         // do nothing if credit already exists
600         return;
601     }
602
603     creditStates[credit] = true;
604     creditContracts.push(credit);
605
606     emit SyncCreditContract(_validator, credit, toRemove);
607 }
```

Listing 2.2: contracts/ListaStakeManager.sol

```
963 /// @dev Returns the total amount of BNB in all validators
964 function getTotalBnbInValidators() public view override returns (uint256) {
965     uint256 totalBnb = 0;
966     for (uint256 i = 0; i < creditContracts.length; i++) {
967         IStakeCredit credit = IStakeCredit(creditContracts[i]);
968         if (creditStates[address(credit)]) {
969             totalBnb += credit.getPooledBNB(address(this)) + credit.lockedBNBs(address(this), 0)
970             ;
971         }
972     }
973     return totalBnb;
974 }
```

```
970     }
971 }
972 return totalBnb;
973 }
```

Listing 2.3: contracts/ListaStakeManager.sol

```
905 function compoundRewards() external override whenNotPaused onlyRole(BOT) {
906     if (totalDelegated == 0) revert ErrorsLib.NotEnoughBnb();
907
908     uint256 totalBNBInValidators = getTotalBnbInValidators();
```

Listing 2.4: contracts/ListaStakeManager.sol

Impact The function `compoundRewards()`, which invokes the function `getTotalBnbInValidators()`, may suffer a temporary DoS issue due to adding the zero address to the array `creditContracts`.

Suggestion Add zero address checks for the variable `credit` in the function `syncCredits()`.

2.1.3 Lack of whitelist checks for the input `dstValidator` in the function `redelegate()`

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `ListaStakeManager`, the function `redelegate()` enables `BOTs` to redelegate `BNBs`. However, the function `redelegate()` does not validate the input `dstValidator`. As a result, `BNBs` may be redelegated to non-whitelisted validators.

```
229 function redelegate(address srcValidator, address dstValidator, uint256 _amount)
230     external
231     override
232     whenNotPaused
233     onlyRole(BOT)
234 {
235     if (srcValidator == dstValidator) revert ErrorsLib.InvalidAddress();
236     if (!validators[srcValidator]) revert ErrorsLib.InactiveValidator();
```

Listing 2.5: contracts/ListaStakeManager.sol

Impact `BNBs` may be redelegated to non-whitelisted validators.

Suggestion Add whitelist checks for the input `dstValidator` in the function `redelegate()`.

2.1.4 Lack of using the modifier `whenNotPaused`

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `ListaStakeManager`, the function `delegateVoteTo()` does not use the modifier `whenNotPaused`. As a result, the function `delegateVoteTo()` is accessible when the project is paused.

```
503 function delegateVoteTo(address _delegateTo) external override onlyRole(DEFAULT_ADMIN_ROLE) {
504     if (_delegateTo == address(0)) revert ErrorsLib.ZeroAddress();
```

Listing 2.6: contracts/ListaStakeManager.sol

Impact The function `delegateVoteTo()` can be invoked when the project is paused.

Suggestion Revise the logic accordingly.

2.1.5 Improper return value in the function `binarySearchCoveredMaxIndex()`

Severity Low

Status Confirmed

Introduced by [Version 1](#)

Description In the contract `ListaStakeManager`, the function `binarySearchCoveredMaxIndex()` determines the maximum index in the withdrawal request queue (i.e., the array `withdrawalQueue`) that can be covered based on the input `_bnbAmount`. However, when there are no new withdrawal requests (Line 453), the function `binarySearchCoveredMaxIndex()` returns zero. This design may be misleading to users, as it could imply that the first request is covered rather than indicating the latest covered index.

```
448 function binarySearchCoveredMaxIndex(uint256 _bnbAmount) public view override returns (uint256)
    {
449     require(
450         withdrawalQueue.length != 0 && withdrawalQueue[0].uuid <= nextConfirmedRequestUUID,
451         "No new requests or old requests have not been fully covered"
452     );
453     if (nextConfirmedRequestUUID > withdrawalQueue[withdrawalQueue.length - 1].uuid) {
454         // all requests have been covered
455         return 0;
456     }
```

Listing 2.7: contracts/ListaStakeManager.sol

Impact When there are no new withdrawal requests, the function `binarySearchCoveredMaxIndex()` returns zero, which is misleading.

Suggestion Revise the logic accordingly.

Feedback from the project The project acknowledged this issue and decided to keep the original design since the `if` branch in the function `claimUndelegated()` will not be triggered.

2.1.6 Potential lock of funds

Severity Low

Status Confirmed

Introduced by [Version 1](#)

Description In the contract `ListaStakeManager`, the function `receive()` is designed to redirect incoming BNBs to `redirectAddress` when the sender is neither a registered credit contract nor `redirectAddress`. However, BNBs, which are directly transferred from `redirectAddress`, could not be withdrawn via any functions. As a result, the BNBs are locked in the contract `ListaStakeManager`, leading to potential lock of funds.

```
896 receive() external payable {
897     if ((!creditStates[msg.sender]) && msg.sender != redirectAddress) {
898         AddressUpgradeable.sendValue(payable(redirectAddress), msg.value);
899     }
900 }
```

Listing 2.8: contracts/ListaStakeManager.sol

```
535 function depositReserve() external payable override whenNotPaused onlyRole(DEFAULT_ADMIN_ROLE)
536 {
537     uint256 amount = msg.value;
538     if (amount == 0) revert ErrorsLib.InvalidAmount();
539     totalReserveAmount += amount;
540 }
541
542 /**
543  * @dev Used by Redirect Address to withdraw reserved funds
544  * @param amount - Amount of BNB to withdraw
545  */
546 function withdrawReserve(uint256 amount) external override whenNotPaused onlyRole(
    DEFAULT_ADMIN_ROLE) {
```

Listing 2.9: contracts/ListaStakeManager.sol

Impact Potential lock of funds.

Suggestion Revise the logic accordingly.

Feedback from the project The project acknowledged this issue and decided to keep the original design.

2.2 Recommendation

2.2.1 Remove redundant code

Status Partially Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description There are several unused variables, errors and checks. It is recommended to remove them for better code readability. Specifically, the following code should be removed or revised.

1. Redundant checks.

```
187 if (slisBnbToMint == 0) revert ErrorsLib.InvalidSlisBnbAmount();
```

Listing 2.10: contracts/ListaStakeManager.sol

2. Redundant errors.

```
10    error AlreadyActive();
```

Listing 2.11: contracts/libraries/ErrorsLib.sol

3. Redundant variable definition (i.e., the variable `bnbAmount` in the function `instantWithdraw()` and `return`).

```
290    function instantWithdraw(uint256 _amountInSlisBnb) external whenNotPaused returns (uint256
        bnbAmount) {
291        uint256 withdrawFee = (_amountInSlisBnb * instantWithdrawFeeRate) / TEN_DECIMALS;
292        instantWithdrawFee += withdrawFee;
293
294        uint256 burnAmount = _amountInSlisBnb - withdrawFee;
295        uint256 bnbAmount = convertSnBnbToBnb(burnAmount);
296        if (bnbAmount < minBnb) revert ErrorsLib.AmountTooSmall();
297
298        IERC20Upgradeable(slisBnb).transferFrom(msg.sender, address(this), _amountInSlisBnb);
299
300        // Won't change the exchange rate since `bnbAmount` is calculated based on the current
            exchange rate
301        amountToDelegate -= bnbAmount;
302        ISLisBNB(slisBnb).burn(address(this), burnAmount);
303
304        if (bnbAmount > 0) {
305            AddressUpgradeable.sendValue payable(msg.sender), bnbAmount);
306        }
307
308        emit InstantWithdraw(msg.sender, _amountInSlisBnb, bnbAmount, withdrawFee);
309
310        return bnbAmount;
311    }
```

Listing 2.12: contracts/ListaStakeManager.sol

Suggestion Remove the redundant code.

Clarification from BlockSec The project acknowledged the redundant checks and decided to keep the original design.

2.2.2 Replace improper errors

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `ListaStakeManager`, the function `compoundRewards()` reverts with the error `NotEnoughFee` (Line 910), which is improper. It is recommended to replace the error `NotEnoughFee` with the error `NotEnoughBnb` for better readability.

```
905    function compoundRewards() external override whenNotPaused onlyRole(BOT) {
906        if (totalDelegated == 0) revert ErrorsLib.NotEnoughBnb();
907
908        uint256 totalBNBInValidators = getTotalBnbInValidators();
```

```
909     if (totalBNBInValidators + undelegatedQuota <= totalDelegated) {
910         revert ErrorsLib.NotEnoughFee();
911     }
```

Listing 2.13: contracts/ListaStakeManager.sol

Suggestion Replace improper errors.

2.2.3 Rename the variable `maxBufferSizePct`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The name of the variable `maxBufferSizePct` is misleading, as it can also imply a minimum size of the remaining value of the variable `amountToDelegate`. It is recommended to rename the variable `maxBufferSizePct` (e.g., `bufferSizePct`) for better readability.

```
982     function skipDelegateOrNot(uint256 _amount) public view override returns (bool, uint256,
983         uint256) {
984         uint256 maxBufferSize = (maxBufferSizePct * getTotalPooledBnb()) / TEN_DECIMALS;
985         uint256 newBufferSize = amountToDelegate - _amount;
986         if (maxBufferSize != 0 && newBufferSize <= maxBufferSize) {
```

Listing 2.14: contracts/ListaStakeManager.sol

Suggestion Rename the variable `maxBufferSizePct`.

2.2.4 Add zero value checks

Status Partially Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description It is recommended to add zero value checks for the input of the following functions.

```
668     function removeValidator(address _address) external override onlyRole(DEFAULT_ADMIN_ROLE) {
669         require(!validators[_address], "Validator should be inactive");
670         require(getDelegated(_address) == 0, "Balance is not zero");
671
672         syncCredits(_address, true);
673         delete validators[_address];
674
675         emit RemoveValidator(_address);
676     }
```

Listing 2.15: contracts/ListaStakeManager.sol

```
546     function withdrawReserve(uint256 amount) external override whenNotPaused onlyRole(
        DEFAULT_ADMIN_ROLE) {
```

Listing 2.16: contracts/ListaStakeManager.sol

```
624 function setMinBnb(uint256 _amount) external override onlyRole(DEFAULT_ADMIN_ROLE) {
```

Listing 2.17: contracts/ListaStakeManager.sol

```
682 function setMaxBufferSizePct(uint256 newPct) external override onlyRole(DEFAULT_ADMIN_ROLE) {
```

Listing 2.18: contracts/ListaStakeManager.sol

```
693 function setInstantWithdrawFeeRate(uint256 _instantWithdrawFeeRate)
```

Listing 2.19: contracts/ListaStakeManager.sol

Suggestion Add zero value checks for listed functions.

Clarification from BlockSec The project decided to keep the zero value option in the functions `setMaxBufferSizePct()` and `setInstantWithdrawFeeRate()` for the potential deprecation of the buffer pool feature.

2.2.5 Unify checks in the functions `instantWithdraw()` and `requestWithdraw()`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `ListaStakeManager`, the functions `instantWithdraw()` and `requestWithdraw()` compare users' intentional withdrawal amount (i.e., the variable `bnbAmount` and `bnbToWithdraw`) to the variable `minBnb`. However, the checks are inconsistent (i.e., `> minBnb` and `>= minBnb`). It is recommended to unify the checks for consistency.

```
290 function instantWithdraw(uint256 _amountInSlisBnb) external whenNotPaused returns (uint256
    bnbAmount) {
291     uint256 withdrawFee = (_amountInSlisBnb * instantWithdrawFeeRate) / TEN_DECIMALS;
292     instantWithdrawFee += withdrawFee;
293
294     uint256 burnAmount = _amountInSlisBnb - withdrawFee;
295     uint256 bnbAmount = convertSnBnbToBnb(burnAmount);
296     if (bnbAmount < minBnb) revert ErrorsLib.AmountTooSmall();
```

Listing 2.20: contracts/ListaStakeManager.sol

```
251 function requestWithdraw(uint256 _amountInSlisBnb) external override whenNotPaused {
252     if (_amountInSlisBnb == 0) revert ErrorsLib.InvalidSlisBnbAmount();
253
254     uint256 bnbToWithdraw = convertSnBnbToBnb(_amountInSlisBnb);
255     if (bnbToWithdraw <= minBnb) revert ErrorsLib.AmountTooSmall();
```

Listing 2.21: contracts/ListaStakeManager.sol

Suggestion Unify checks in the functions `instantWithdraw()` and `requestWithdraw()`.

2.3 Note

2.3.1 The project should set the variable `instantWithdrawFeeRate` properly

Introduced by [Version 1](#)

Description In the contract `ListaStakeManager`, the function `compoundRewards()` accumulates profits for users by updating the exchange rate between `BNB` and `slisBNB` (i.e., increases the price of `slisBNB`). The function `instantWithdraw()` allows users to perform an instant withdrawal by paying the withdrawal fee (i.e., the variable `withdrawFee`). However, this design may introduce potential arbitrage opportunities if the variable `instantWithdrawFeeRate` is improperly configured. Specifically, a user could first invoke the function `deposit()` before the invocation of `compoundRewards()`, thereby minting the `slisBNB` at a lower price. The user could then invoke the function `instantWithdraw()` after the invocation of `compoundRewards()`, thereby selling the `slisBNB` at a higher price with the withdrawal fee. If the withdrawal fee is smaller than the profit, the user could have the chance to perform arbitrage. The project must ensure the variable `instantWithdrawFeeRate` is properly configured to prevent potential arbitrage opportunities.

```

905  function compoundRewards() external override whenNotPaused onlyRole(BOT) {
906      if (totalDelegated == 0) revert ErrorsLib.NotEnoughBnb();
907
908      uint256 totalBNBInValidators = getTotalBnbInValidators();
909      if (totalBNBInValidators + undelegatedQuota <= totalDelegated) {
910          revert ErrorsLib.NotEnoughFee();
911      }
912      uint256 totalProfit = totalBNBInValidators + undelegatedQuota - totalDelegated;
913      uint256 fee = SLisLibrary.calculateFeeFromDailyProfit(totalProfit, synFee, TEN_DECIMALS);
914
915      totalDelegated += totalProfit;

```

Listing 2.22: contracts/ListaStakeManager.sol

```

290  function instantWithdraw(uint256 _amountInSlisBnb) external whenNotPaused returns (uint256
      bnbAmount) {
291      uint256 withdrawFee = (_amountInSlisBnb * instantWithdrawFeeRate) / TEN_DECIMALS;
292      instantWithdrawFee += withdrawFee;
293
294      uint256 burnAmount = _amountInSlisBnb - withdrawFee;
295      uint256 bnbAmount = convertSnBnbToBnb(burnAmount);
296      if (bnbAmount < minBnb) revert ErrorsLib.AmountTooSmall();
297
298      IERC20Upgradeable(slisBnb).transferFrom(msg.sender, address(this), _amountInSlisBnb);
299
300      // Won't change the exchange rate since `bnbAmount` is calculated based on the current
      exchange rate
301      amountToDelegate -= bnbAmount;
302      ISLisBNB(slisBnb).burn(address(this), burnAmount);
303
304      if (bnbAmount > 0) {
305          AddressUpgradeable.sendValue(payable(msg.sender), bnbAmount);
306      }

```



```
307
308     emit InstantWithdraw(msg.sender, _amountInSlisBnb, bnbAmount, withdrawFee);
309
310     return bnbAmount;
```

Listing 2.23: contracts/ListaStakeManager.sol

Feedback from the project The project stated that they will adjust the `instantWithdrawFeeRate` variable accordingly to encourage or mitigate the potential arbitrage.

2.3.2 Ensure the rationality of the variable `instantWithdrawFeeRate`

Introduced by Version 1

Description In the contract `ListaStakeManager`, the function `skipDelegateOrNot()` is invoked in the function `delegateTo()` to check the available delegating amount before delegations. If the new buffer size (i.e., the variable `newBufferSize`) is smaller than the max buffer size determined based on the variable `maxBufferSizePct` and the pooled `BNB` amount, the delegation fails. It is worth noting that this design allows a malicious user to front-run delegations (i.e., the invocation of the function `delegateTo()`) by performing instant withdrawals (i.e., the invocation of the function `instantWithdraw()`). As a result, delegations may revert due to the malicious instant withdrawals. The project should ensure the rationality of the variable `instantWithdrawFeeRate`.

```
202     function delegateTo(address _validator, uint256 _amount) external override whenNotPaused
203         onlyRole(BOT) {
204         if (_amount > amountToDelegate) revert ErrorsLib.NotEnoughBnb();
205
206         if (!validators[_validator]) revert ErrorsLib.InactiveValidator();
207         if (_amount < IStakeHub(STAKE_HUB).minDelegationBNBChange()) {
208             revert ErrorsLib.InvalidAmount();
209         }
210
211         (bool _skipDelegate,,) = skipDelegateOrNot(_amount);
212         if (_skipDelegate) {
213             revert ErrorsLib.BufferTooSmall();
214         }
```

Listing 2.24: contracts/ListaStakeManager.sol

```
982     function skipDelegateOrNot(uint256 _amount) public view override returns (bool, uint256,
983         uint256) {
984         uint256 maxBufferSize = (maxBufferSizePct * getTotalPooledBnb()) / TEN_DECIMALS;
985         uint256 newBufferSize = amountToDelegate - _amount;
986
987         if (maxBufferSize != 0 && newBufferSize <= maxBufferSize) {
988             // max buffer size is not exceeded, do not delegate
989             return (true, maxBufferSize, amountToDelegate);
990         } else {
991             return (false, maxBufferSize, amountToDelegate);
992         }
```

Listing 2.25: contracts/ListaStakeManager.sol

```
290 function instantWithdraw(uint256 _amountInSlisBnb) external whenNotPaused returns (uint256
    bnbAmount) {
291     uint256 withdrawFee = (_amountInSlisBnb * instantWithdrawFeeRate) / TEN_DECIMALS;
292     instantWithdrawFee += withdrawFee;
293
294     uint256 burnAmount = _amountInSlisBnb - withdrawFee;
295     uint256 bnbAmount = convertSnBnbToBnb(burnAmount);
296     if (bnbAmount < minBnb) revert ErrorsLib.AmountTooSmall();
297
298     IERC20Upgradeable(slisBnb).transferFrom(msg.sender, address(this), _amountInSlisBnb);
299
300     // Won't change the exchange rate since `bnbAmount` is calculated based on the current
        exchange rate
301     amountToDelegate -= bnbAmount;
```

Listing 2.26: contracts/ListaStakeManager.sol

Feedback from the project The project stated that they will adjust the `instantWithdrawFeeRate` variable accordingly to prevent malicious users from front-running delegations.

2.3.3 Contract deployment chains

Introduced by [Version 1](#)

Description The audited contracts will (only) be deployed on the [BSC](#) chain.

2.3.4 Potential centralization risks

Introduced by [Version 1](#)

Description In this project, several privileged roles (e.g., the role [BOT](#)) can conduct sensitive operations, which introduces potential centralization risks. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.3.5 OpenZeppelin initializable upgrade migration risks

Introduced by [Version 1](#)

Description The project currently uses OpenZeppelin's Initializable contract (v4.8.3) to implement upgradeable contracts. It is important to note that Initializable versions v5.0.0 and later introduce ERC-7201 namespaced storage to mitigate storage collision risks. This change relocates initialization state variables from direct storage slots (e.g., `_initialized`) to namespaced storage structures (e.g., `$_initialized`). When upgrading to newer Initializable versions, the project must ensure proper migration of initialization state to prevent contracts from being reinitialized, which could lead to severe security vulnerabilities, including state corruption and unauthorized access.

2.3.6 Proxy deployment and implementation binding should be atomic

Introduced by [Version 1](#)

Description To prevent potential front-running attacks, it is recommended that proxy deployment and implementation binding be executed atomically within a single transaction. If these operations are performed separately, malicious actors could exploit the time window between deployment and binding to front-run the implementation binding transaction. An attacker could potentially bind their own malicious implementation to the newly deployed proxy contract, gaining unauthorized control over the protocol's upgrade mechanism and user funds. This race condition poses significant security risks, including complete protocol compromise, fund theft, and unauthorized access to privileged functions.

2.3.7 Ensure that all BOTs are working properly

Introduced by [Version 1](#)

Description In the project, the role [BOT](#) is responsible for executing critical operations such as delegations and reward compounding. The project must ensure that all [BOTs](#) are working properly.

```
925 function _burnRefundSlisBnb() private {
926     if (refund.remainingSlisBnb == 0) return;
927     if (block.timestamp / 1 days < (refund.lastBurnTime / 1 days + 1)) return; // burn once a
        day
928
929     uint256 burnAmount = refund.dailySlisBnb;
930     if (burnAmount > refund.remainingSlisBnb) {
931         burnAmount = refund.remainingSlisBnb;
932         refund.remainingSlisBnb = 0;
933         refund.dailySlisBnb = 0;
934     } else {
935         refund.remainingSlisBnb -= burnAmount;
936     }
937
938     refund.lastBurnTime = block.timestamp;
939     ISLisBNB(slisBnb).burn(address(this), burnAmount);
940 }
```

Listing 2.27: contracts/ListaStakeManager.sol

